

CryptAnalysis Project Report

Ethan Terrero, Miles Anderson, and Noel Hernandez

Abstract

Modern encryption tools often act as "black boxes," hiding the actual mechanics of cryptography from the end user. For our project, we built a fully functional C++ Command Line Interface (CLI) utility designed to bridge the gap between how and why various encryption methods work. We built a system that not only encrypts files using two of the most popular methods, AES-256 (a block cipher) and ChaCha20 (a stream cipher), but also analyzes the data to help users understand why it is secure. To evaluate the encrypted files, we used three different statistical tests: the Avalanche Effect to measure data diffusion, Shannon Entropy to verify structural randomness, and Chi-Squared analysis to check for uniform byte frequencies. By calculating these metrics, we are able to demonstrate the differences in cryptographic strength and pattern leakage across various algorithms.

Keywords

Cryptanalysis, AES-256, ChaCha20, Block Cipher, Stream Cipher, Avalanche Effect, Shannon Entropy, Chi-Squared Analysis

Team Intro

The development of our tool was a highly collaborative effort, with all three team members working across the entire project. Since Noel had the strongest background in cryptography, he took the lead on the analysis side, writing the code to implement the Avalanche Effect, Shannon Entropy, and Chi-Squared statistical tests, and validating that the numerical results matched the expected theoretical behavior. Ethan and Miles focused primarily on the core encryption mechanics and the software infrastructure. They integrated the OpenSSL library to get the AES-256 and ChaCha20 algorithms implemented, and built the file I/O system to handle reading and writing the data, as well as the CLI interface and benchmarking harness. Ultimately, we all shared the work of debugging the application, running the performance benchmarks, tuning parameters (like test file sizes and number of runs), and putting together the final project presentation and report.

Related Work

While researching for our project, we broke down cryptographic softwares into three general categories: practical command line encryption utilities, educational cryptography platforms, and rigorous statistical testing suites. Tools like GnuPG and the OpenSSL utility are great for securing sensitive data, but serve just as encryption tools, forcing users to blindly trust the underlying algorithms without providing any feedback on the integrity or diffusion of the ciphertext. Educational platforms such as CrypTool, address this by providing interactive environments for exploring cryptographic concepts, but they often act as isolated sandboxes that are unable to be used for day to day encryption. For evaluating the mathematical “strength” of ciphers, researchers use highly complex software such as NIST Statistical Test Suite or Dieharder. While these are the gold standard for testing pseudorandomness, they are quite in depth, with a steep learning curve, and don’t have built in encryption. Requiring users to manually generate and encrypt their text. Our goal was to design a software that bridged the gap between the three. By integrating the core file I/O and cryptography of a standard utility with the mathematical transparency of an educational testing suite, our tool allows users to securely encrypt their data and immediately evaluate its cryptographic strength using built-in metrics. While not as in depth as some high level math suites, these algorithms provide a strong general sense of what each algorithm is doing.

Main Body of Work

Design

We split the project into six modules, each handling one thing. The crypto/ folder has our cipher implementations, with two classes, AesCipher and ChaChaCipher, both inheriting from an abstract Encryptor superclass. Encryptor defines a shared interface with encrypt() and decrypt(), so the CLI and benchmarking code can work through that abstraction without caring which cipher is actually running underneath. That also means adding a new algorithm later is straightforward.

The io/ folder takes care of file reading and writing, loading plaintext into memory, writing ciphertext to disk, and handling I/O errors. The analysis/ folder has three independent cryptographic tools: Shannon entropy, chi-squared byte frequency analysis, and avalanche effect testing. Each one takes plaintext or ciphertext and returns a numeric result (entropy in bits/byte, a chi-squared statistic with p-value, or average percentage of bits flipped).

We also have a benchmark/ module with a CPU cycle counter (where supported) and a wall-clock timer via std::chrono. It defines a simple harness that runs encryption and decryption multiple times on test files and computes average times and throughput. The utils/ folder holds SHA-256 hashing for integrity checks and miscellaneous helper functions used across the codebase.

The other big design choice was our self-describing file header. Every encrypted file starts with a header containing the algorithm identifier (like AES-256 or ChaCha20), cipher mode (ECB, CBC, or stream), salt, and IV or nonce, all in a fixed binary layout. When decrypting, the CLI reads that header, reconstructs the right cipher object, derives key material from the user's passphrase using the stored salt,

and applies the correct algorithm and mode automatically. The user doesn't have to remember what settings they encrypted with. This cuts down on user error and makes it easy to run the same analysis routines across ciphertexts from different algorithms in a consistent way.

Algorithm

AES-256 is a symmetric block cipher, meaning the same key handles both encryption and decryption. It's the current encryption standard and shows up in virtually every security protocol, from HTTPS to disk encryption. It works on 128-bit blocks and runs through a series of substitution and permutation rounds to provide confusion and diffusion. We focused on two common modes of operation: Electronic Codebook (ECB) and Cipher Block Chaining (CBC).

In ECB mode, each plaintext block is encrypted independently with the same key. It's simple and fast, but generally insecure for real-world data because identical plaintext blocks produce identical ciphertext blocks, which leaks patterns in the input. CBC mode fixes this by XORing each plaintext block with the previous ciphertext block before encrypting, with the first block XORed against an initialization vector. That chaining means even if two plaintext blocks are identical, their ciphertexts will differ as long as their preceding contexts differ, which greatly improves diffusion and hides repeated patterns.

We also implemented ChaCha20, a modern stream cipher that serves as the primary alternative to AES in many protocols (for example, TLS uses ChaCha20-Poly1305). ChaCha20 generates a pseudorandom keystream from a key, nonce, and counter, then XORs that keystream with the plaintext to produce ciphertext. Since it operates as a stream cipher, flipping a bit in the plaintext directly flips the corresponding bit in the ciphertext, which has implications for how we interpret avalanche-style tests. Comparing AES-256 in ECB and CBC with ChaCha20 lets us look at the tradeoffs between block ciphers and stream ciphers in terms of performance, diffusion, and resistance to simple pattern analysis.

Implementation

Why OpenSSL

Implementing AES and ChaCha20 from scratch would be extremely difficult to do. A naive implementation would most likely fail our own tests. Instead, we opted to use OpenSSL which would provide us the necessary encryption algorithms, was open-source, battle-tested and used in industry. OpenSSL handles the low-level cryptographic functionality which is the actual AES block operations and the ChaCha20 keystream generation.

How we use it

OpenSSL's EVP (envelope) API is the recommended high-level interface for symmetric encryption. The general pattern starts with creating a cipher context (EVP_CIPHER_CTX) and initializing it with the chosen algorithm, key, and IV or nonce. An EVP_CIPHER object describes the

algorithm and mode (like AES-256-CBC), while the `EVP_CIPHER_CTX` holds the live encryption state, including any buffered partial blocks.

For encryption, we call `EVP_EncryptInit_ex` to configure the context, then repeatedly call `EVP_EncryptUpdate` with chunks of plaintext, which produces chunks of ciphertext. Once all the input has been processed, we call `EVP_EncryptFinal_ex` to flush any remaining buffered bytes, including padding for block modes. Decryption follows the same structure with `EVP_DecryptInit_ex`, `EVP_DecryptUpdate`, and `EVP_DecryptFinal_ex`. When we're done, we free the context with `EVP_CIPHER_CTX_free`.

Key and IV/nonce material come from user-supplied passphrases run through a standard key derivation function (via OpenSSL's utilities), combined with a randomly generated salt that gets stored in the file header. This means using the same passphrase on different files still produces different keys, and IVs/nonces don't get reused. OpenSSL's documentation and example code were really helpful for making sure we were using the EVP API correctly and securely.

Evaluation Methodology

To properly benchmark and compare our algorithms, we built a dedicated analysis layer that runs directly against the generated ciphertexts. We didn't just want to test speed; we wanted to evaluate the structural integrity and security-relevant properties of the encrypted output. We implemented three statistical methods (avalanche effect, Shannon entropy, and chi-squared analysis) alongside a performance benchmarking harness.

We ran everything on a macOS system (Darwin 25.3.0) with a modern multi-core CPU and enough memory to load roughly 100 MB test files into RAM. For performance benchmarks, we created a ~100 MB input file with pseudo-random and mixed content, then encrypted and decrypted it using each algorithm/mode combination: AES-256-ECB, AES-256-CBC, and ChaCha20. We repeated the encrypt/decrypt cycle multiple times per configuration (5 runs) and averaged the wall-clock times to smooth out noise from background processes. We also measured ciphertext size to quantify overhead from headers and padding.

For the statistical tests, we used ciphertexts generated from similar-sized files and ran entropy and chi-squared analyses over the full ciphertext byte stream. The avalanche test is more computationally intensive, so we restricted that analysis to the first 256 bit positions in the plaintext. We considered an avalanche result "good" if the average percentage of flipped ciphertext bits from a single plaintext bit flip fell between roughly 45% and 55%, which is consistent with the Strict Avalanche Criterion for a strong block cipher.

We intentionally separate two kinds of evaluation: performance (time and throughput) and statistical behavior (diffusion and randomness). Our threat model is a passive attacker who can observe ciphertext and try simple pattern or frequency-based attacks. We're not attempting full cryptanalytic attacks or side-channel analysis. Instead, we're focused on whether the ciphertext looks random enough and whether small changes in plaintext produce large, unpredictable changes in ciphertext, which is what you'd expect from a secure scheme.

We implemented three primary statistical methods:

Avalanche:

The Strict Avalanche Criterion (SAC) is a method of testing that ensures high security. SAC ultimately provides a metric for diffusion. We flip one bit in the plaintext, and this one single change should have a massive cascading effect in the output ciphertext. If we simply had a one-to-one change in plaintext, which could be visibly seen in the ciphertext, attackers could exploit this leakage of internal structure to break the encryption. We use this to test our AES-256 algorithm which is a symmetric block cipher.

SAC ensures that for one bit flip in the plaintext, at least ~50% of the ciphertext changes as a result. Our program encrypts the original plaintext as a baseline for comparison. We focus on the first 256 bit positions in the plaintext which is the first 32 bytes (2 blocks x 16 bytes). Using only the first two blocks for testing came down to issues regarding computation. If we were given a 1MB file, there are approximately 8 million bit positions. This means we would have to run a test for every single bit position, then aggregate that data. Running 8 million AES encryptions would be too costly. Just the first two blocks are representative enough because of AES internal round functions. Since each round does the same operations (SubBytes, ShiftRows, MixColumns, AddRoundKey), we will see good diffusion in the first two blocks, it will be seen in the remaining blocks as well.

We conducted this test by encrypting a baseline file. In a loop (up to 256). With a copy of the plaintext, we would flip a single bit there, then store it into a `uint_8` vector. Our goal is to focus on changes in the raw bytes, and some of these changes can't be typecasted to a valid character. In every iteration of the loop, once we get the new encrypted result from the bit flip, we XOR that with the baseline ciphertext. The XOR operation produces 1s where bits differ and 0s where they are the same. We record the percentage changed for that one run, then repeat this process until we have all 256 statistics. At the end, we return the average percentage changed and compare that to our ~45% to ~55% cutoff.

Shannon Entropy

Shannon entropy provides a measure for how unpredictable or “surprising” a message is. In the context of cryptography, this measure represents how many yes/no questions an attacker would need to ask to guess the next byte in the sequence. Here, knowledge and entropy are opposites. The more patterns that are identifiable in the ciphertext, the more information an attacker has to narrow down the number of guesses needed to break the encryption algorithm. For this project, the maximum possible entropy is 8 since we measure entropy at the byte level. An individual byte is 8 bits wide which yields 2^8 , or 256 possibilities. Maximum entropy means each byte is as equal as possible which forces an attacker to brute force every possibility. This renders frequency analysis and pattern based attacks completely useless since there is no identifiable structure in the ciphertext that could be used to reverse engineer a solution.

To measure entropy in our project, we take in the ciphertext as a byte vector and count how many times each of the 256 possible byte values (0x00-0xFF) appears. Each count is then divided by the total number of bytes to produce a probability $p(x)$ for that value. We apply the Shannon entropy formula by iterating over all 256 values, skipping any values that don't appear, and then accumulating the result. This function also tracks the most frequently appearing byte and the least frequently appearing byte which are used to indicate how close the distribution is to the ideal frequency. This differs from our frequency analysis because entropy asks how much information is packed into the byte stream. It converts the counts to probabilities and feeds them to the Shannon entropy formula which produces a single score

between 0-8. That single score represents the average number of bits of surprise per byte in the ciphertext. We compare the final value calculated against two thresholds. A score of 7.9 or above is considered a pass while a score below 7.5 flags detectable patterns which suggests the ciphertext is leaking structure from the plaintext.

$$H = - \sum p(x) \log p(x)$$

We wanted a mathematical way to prove when an encryption mode is leaking patterns. This metric is perfect for catching the structural flaws in modes like ECB, where identifiable patterns clearly reduce the randomness of the final output.

Chi-Squared

Chi-squared is about comparing our expected outcomes against what we observe in the real world. A simple example of this is a coin toss. We would expect 50% heads, and 50% tails out of 100 tosses. However, we will see some variation in the observed outcomes. What we want to know is how much variation is attributed to randomness, versus some other factor. If we did 100 coin tosses in a row, and we got 90 heads and 10 tails we would have to ask ourselves if that's too extreme.

In the context of cryptography, our expected value is an even distribution of every possible symbol in the ciphertext versus the real observed distribution of symbols in the ciphertext. Our expected value is an even appearance of every symbol since that leaks the least amount of structure from the plaintext to the ciphertext. It makes identifying patterns virtually impossible. We divide the total size of the actual ciphertext by 256. This gives us the expected count for each symbol if it's even across the output. For each byte value that appears, we take that value as an index into an array of size 256 and increment the count for that. This works since the byte can be read as a number in the range of 0 to 255. On every iteration, we calculate the $(\text{observed} - \text{expected})^2 / \text{expected}$. We accumulate this across all 256 values.

Our final result is the chi-statistic. The problem is we want to compare this to a p-value from a normal distribution which requires converting the chi-statistic to a z-score. We do this by plugging our result into Wilson-Hilferty's equation for standardizing the chi-squared results rather than including an additional dependency to include a separate mathematical function for getting the area under the Chi-squared distribution. We can use C++'s `<cmath>` which includes the `std::erfc` function. This function takes in a z-score and gives you the area in the tail of the normal distribution beyond that point. Our return value is between 0-2 so we scale this down to a 0-1 range by multiplying by 0.5 which is now our p-value. We can compare this against our thresholds. If our p-value is less than 0.05, we fail. This means the deviation from uniform is so large it is statistically unlikely to be random chance, meaning our

ciphertext has detectable patterns in it. If our p-value is greater than 0.05, there isn't enough evidence to reject randomness which means we pass. In the situation where we have a p-value greater than 0.95, we take note of that since we're expecting some natural variation. In a real world example, we wouldn't expect every byte to appear almost exactly the same number of times.

$$\chi^2 = \sum \frac{(O - E)^2}{E}$$

Where O is the observed frequency and E is the expected frequency.

Results and Analysis

We measured both performance results, and cryptographic strength in our testing.

Performance

We first tested the raw processing speed and file size overhead. The table below shows the results of encrypting and decrypting a roughly 100MB file.

Algorithm	Mode	Enc (MB/s)	Dec (MB/s)	Enc (s)	Dec (s)	Overhead	Out Size
aes256	cbc	129.47	146.58	0.772376	0.682226	0.0%	104857652
aes256	ecb	150.66	150.29	0.663734	0.665396	0.0%	104857636
chacha20	none	135.70	143.74	0.736936	0.695680	0.0%	104857636

While AES-256 ECB had the fastest encryption time, this speed comes at the cost of severe security vulnerabilities (as shown in the later tests). AES-256 CBC was slightly slower due to the overhead of XORing each block with the previous ciphertext. ChaCha20 performed exceptionally well as a stream cipher, providing high speed encryption with zero byte overhead compared to the original file size.

Avalanche Effect (Diffusion)

```
[ DATA ] Avalanche Effect (AES-256-CBC):  
Bits Tested: 128  
Min Bit Change: 45.625%  
Max Bit Change: 57.1875%  
Avg Bit Change: 49.6643% (~50% IDEAL)
```

```
[ DATA ] Avalanche Effect (ChaCha20):  
Bits Tested: 64  
Avg Bit Change: 0.195312% (STREAM CIPHER EXP)
```

Above are the results from our tests. We flipped a single bit in the plaintext and measured the impact on the ciphertext. An ideal block cipher should flip about 50% of the output bits. AES-256 CBC achieved an average bit change of 49.6643%, showing excellent diffusion. ChaCha20, however, only showed 0.195312% change. This demonstrates how stream ciphers work. Since the keystream is directly XORed against the plaintext, flipping one bit in the file strictly flips that exact corresponding bit in the encrypted output.

Shannon Entropy

```
=====
[ DATA ] Shannon Entropy (ChaCha20):  
Bytes Analyzed: 65572  
Calculated Entropy: 7.99712 bits/byte (Ideal: ~8.0)  
=====
```

```
=====
[ DATA ] Shannon Entropy (AES-256-ECB):  
Bytes Analyzed: 65572  
Calculated Entropy: 4.00687 bits/byte (WARNING: LOW)  
=====
```

```
=====
[ DATA ] Shannon Entropy (AES-256-CBC):  
Bytes Analyzed: 65588  
Calculated Entropy: 7.9966 bits/byte (Ideal: ~8.0)  
=====
```

Above are the results for our entropy testing. Both ChaCha20 and AES-256 CBC scored nearly perfect 8.0 values, meaning the data is highly random. However, AES-256 ECB scored a dangerously low 4.00687 bits/byte. Because ECB mode encrypts identical plaintext blocks into identical ciphertext blocks using the same key, it leaves massive patterns in the data, drastically reducing the overall entropy; we see a similar pattern in the Chi-Squared test.

Chi-Squared

```
=====
[ DATA ] Chi-Squared Analysis (AES-256-ECB):  
Chi-Squared Stat (X^2): 982524  
P-Value: 0 (FAIL: < 0.05)  
=====
```



```
=====
[ DATA ] Chi-Squared Analysis (AES-256-CBC):
Bytes Analyzed: 65588
Chi-Squared Stat (X^2): 241.892
P-Value: 0.712656 (PASS: 0.05 - 0.95)
=====
```

```
=====
[ DATA ] Chi-Squared Analysis (ChaCha20):
Bytes Analyzed: 65572
Chi-Squared Stat (X^2): 244.932
P-Value: 0.663602 (PASS: 0.05 - 0.95)
=====
```

Consistent with our entropy findings, both ChaCha20 and AES-256 CBC passed comfortably with healthy p-values. AES-256 ECB failed spectacularly with a p-value of 0 and an incredibly high raw chi-squared statistic. This shows that specific bytes are appearing far too often, showing the file is highly structured, and prone to frequency attacks.

Issues

While our CLI tool is fully functional, we ran into a few technical hurdles during development. The most complicated issue was our original plan to build a visualization of the encryption process, and their strengths. We ultimately had to pivot away from this because we couldn't find a clean, effective way to render the data visually without overcomplicating the core project scope.

Additionally, there are a few technical limitations we haven't fully resolved yet. Currently, our file buffering strategy restricts the maximum file size we can process to 2GB. We also haven't tested the tool on Windows natively.. Finally, while we initially planned to include a graphical user interface (GUI) and Galois/Counter Mode (GCM) for AES, however we left these out to focus on the statistical testing, opting to stick with a CLI instead.

Conclusions

We believe we achieved our goal of building a functional, secure file encryptor paired with an educational analysis tool. By combining real production grade algorithms like AES-256 and ChaCha20 with statistical testing, our tool shows the real performance and security differences between block and stream ciphers. The project allowed us to deliver a complete C++ application with robust file I/O, OpenSSL integration, and an automated benchmarking suite, providing both a secure utility and a strong foundation for future cryptographic experiments.

Lessons learned

Building this project really bridged the gap between how some of these algorithms work in theory and why they are effective in practice. It is one thing to read that a mode like AES-ECB is insecure, but it is a completely different experience to build a program that mathematically proves it by watching the ciphertext fail entropy and chi-squared tests in real time. Implementing the Avalanche test forced us to think carefully about what diffusion actually means and how to quantify it over real files.

We also got to see the practical tradeoffs of these algorithms firsthand. Our benchmarking showed how fast and efficient a stream cipher like ChaCha20 can be, but our Avalanche tests highlighted exactly where ChaCha20, used without authentication, allows bit-flip manipulations that a block cipher in a chaining mode would diffuse. This clarified why modern protocols almost always pair ChaCha20 with a MAC (for example, in ChaCha20-Poly1305) and why “confidentiality only” is not enough for real-world security.

On the software engineering side, integrating OpenSSL’s EVP API, designing a modular architecture, and building a self-describing file format taught us a lot about how cryptographic libraries are used in production code. In the end, this project was a great way to bridge the gap between theoretical cryptography and practical systems programming, and it gave us a much deeper intuition for both the strengths and the pitfalls of modern encryption schemes.

References

Related softwares:

- <https://www.gnupg.org/>
- <https://csrc.nist.gov/projects/random-bit-generation/documentation-and-software>
- <https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-22r1a.pdf>
- <https://webhome.phy.duke.edu/~rgb/General/dieharder.php>
- <https://www.cryptool.org/en/cto/>

Avalanche Testing resources:

- <https://www.geeksforgeeks.org/computer-networks/avalanche-effect-in-cryptography/>
- [https://en.wikipedia.org/wiki/Avalanche_effect#:~:text=In%20cryptography%2C%20the%20avalanche%20effect,half%20the%20output%20bits%20flip\).](https://en.wikipedia.org/wiki/Avalanche_effect#:~:text=In%20cryptography%2C%20the%20avalanche%20effect,half%20the%20output%20bits%20flip).)

AES-256/ChaCha20 resources:

- <https://www.youtube.com/watch?v=O4xNJstN6E>
- <https://www.youtube.com/watch?v=Uelpq-C-GSA>
- <https://www.geeksforgeeks.org/computer-networks/advanced-encryption-standard-aes/>
- <https://en.wikipedia.org/wiki/ChaCha20-Poly1305>
- https://en.wikipedia.org/wiki/Advanced_Encryption_Standard

Shannon Entropy resources:

- <https://www.youtube.com/watch?v=9r7FIXEAGvs>
- [https://en.wikipedia.org/wiki/Entropy_\(information_theory\)](https://en.wikipedia.org/wiki/Entropy_(information_theory))
- <https://www.youtube.com/watch?v=R4OIXb9aTvQ&t=185s>

Chi-Squared resources:

- <https://ibmathsresources.com/2014/06/15/using-chi-squared-to-crack-codes/>

- <https://sites.google.com/site/cryptocrackprogram/user-guide/statistics-tab/chi-square>
- <http://www.practicalcryptography.com/cryptanalysis/text-characterisation/chi-squared-statistic/>

OpenSSL documentation:

- <https://docs.openssl.org/master/man3/>